

Universidad Nacional Autónoma de México
Facultad de Ingeniería

Sistemas Distribuidos — Semestre 2027-1

**Laboratorio S07: Mexi Banco: de un monolito a
servicios que se encuentran solos**

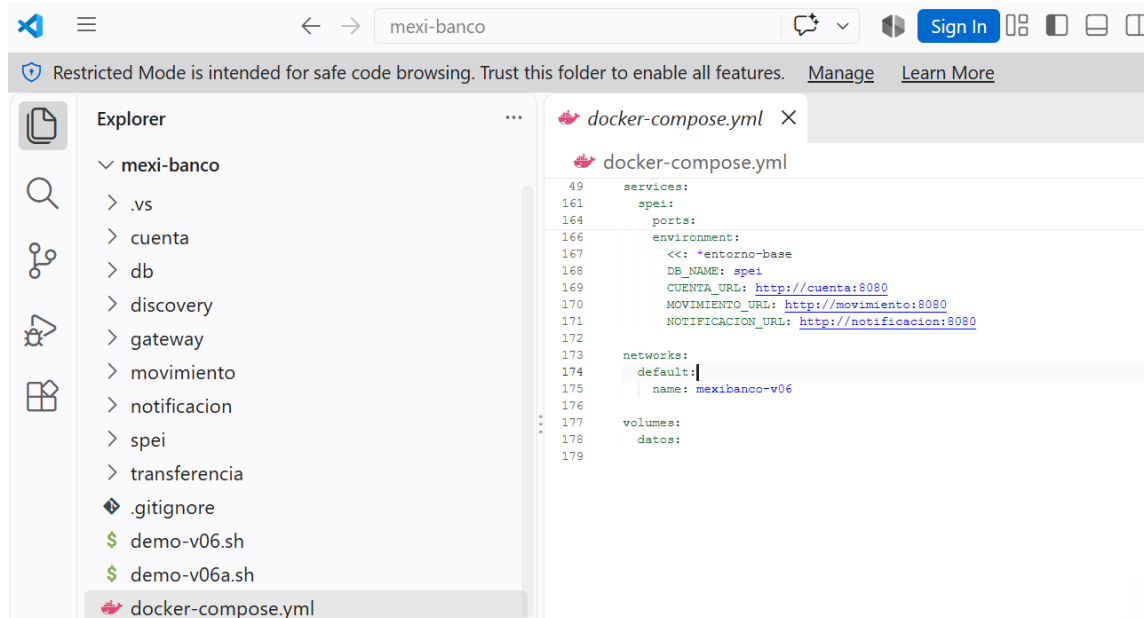
Alumna: Cecilia Ximena Solís Cisneros
Fecha de entrega: 09 de octubre de 2026

Paso 1. La historia.

En el monolito, transferencia y cuenta compartían una base de datos. El cargo y el abono se realizaban dentro de una misma transacción: si el abono fallaba, también se deshacía el cargo.

Al dividir el banco en cinco servicios, las llamadas entre transferencia y cuenta pasaron a realizarse mediante HTTP. Cada servicio confirma sus propias operaciones, por lo que un cargo puede quedar registrado aunque después falle el abono. La transacción local ya no garantiza que ambas operaciones se completen juntas.

En `docker-compose.yml` encontré `CUENTA_URL`: <http://cuenta:8080>. Esta dirección está escrita de forma fija. El laboratorio incorpora un directorio para localizar servicios, un gateway como entrada común y un balanceador para repartir solicitudes entre varias copias.



2.0 · Antes de tocar nada: ¿cómo está hoy?

Levanté el proyecto y comprobé su estado con `docker compose ps`. Los seis contenedores estaban healthy: la base de datos y los cinco servicios. Al abrir <http://localhost:8081/cuentas/000>, cuenta respondió con un error 404 porque la CLABE 000 no existía. En cambio, <http://localhost:8761> no cargó porque el directorio todavía no estaba habilitado.

```

cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose up -d
[+] up 7/7
✓ Network mexibanco-v06          Created
✓ Container mexi-banco-v06-db-1   Healthy
✓ Container mexi-banco-v06-cuenta-1 Started
✓ Container mexi-banco-v06-movimiento-1 Started
✓ Container mexi-banco-v06-transferencia-1 Started
✓ Container mexi-banco-v06-notificacion-1 Started
✓ Container mexi-banco-v06-spei-1  Started

cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)

```

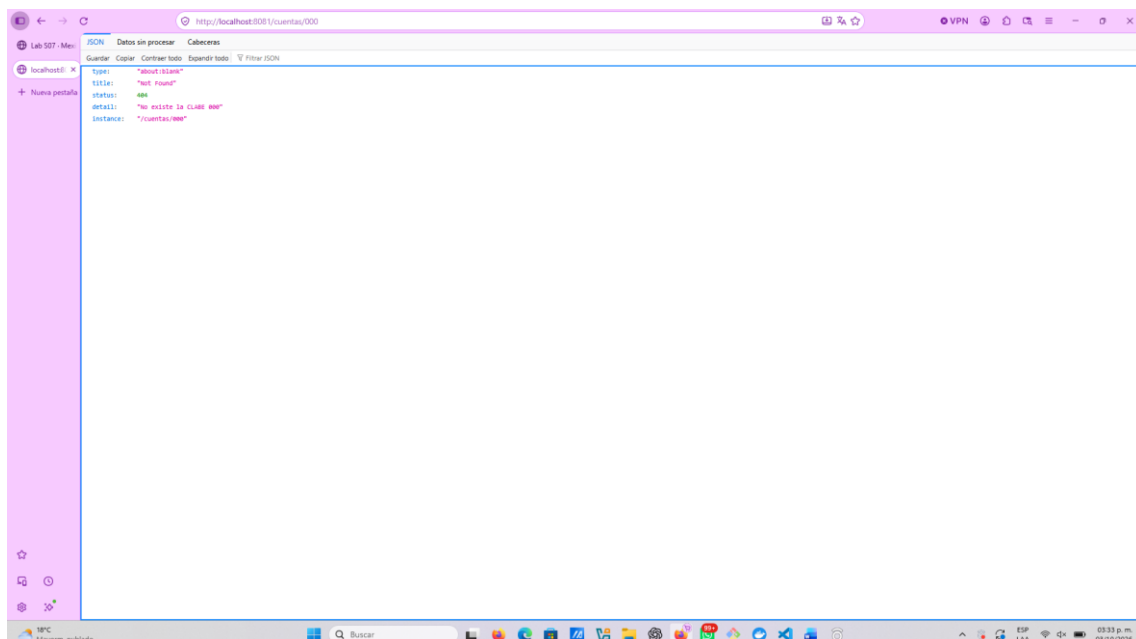
```

cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose ps

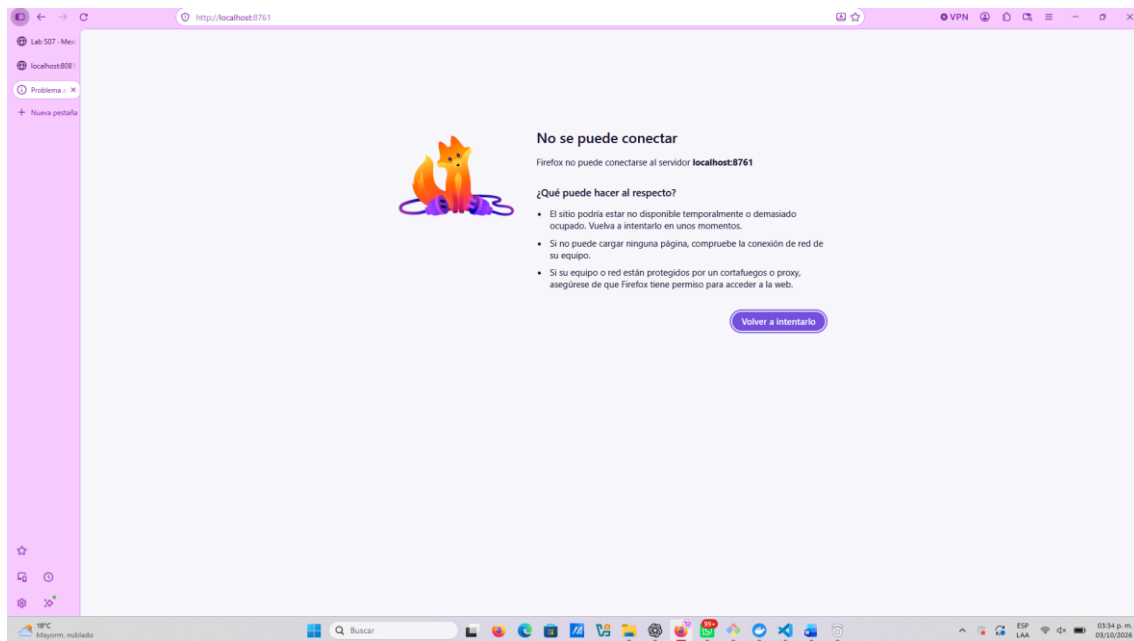
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
mexi-banco-v06-cuenta-1	mexi-banco-v06-cuenta	"java -jar app.jar"	cuenta	About a minute ago	Up About a minute (healthy)	0.0.0.0:8081
mexi-banco-v06-db-1	postgres:17-alpine	"docker-entrypoint.s..."	db	About a minute ago	Up About a minute (healthy)	5432/tcp
mexi-banco-v06-movimiento-1	mexi-banco-v06-movimiento	"java -jar app.jar"	movimiento	About a minute ago	Up About a minute (healthy)	0.0.0.0:8082
mexi-banco-v06-notificacion-1	mexi-banco-v06-notificacion	"java -jar app.jar"	notificacion	About a minute ago	Up About a minute (healthy)	0.0.0.0:8084
mexi-banco-v06-spei-1	mexi-banco-v06-spei	"java -jar app.jar"	spei	About a minute ago	Up About a minute (healthy)	0.0.0.0:8085
mexi-banco-v06-transferencia-1	mexi-banco-v06-transferencia	"java -jar app.jar"	transferencia	About a minute ago	Up About a minute (healthy)	0.0.0.0:8083

<http://localhost:8081/cuentas/000> → cuenta te contesta directo: "no existe la cuenta 000" (un 404 en JSON).

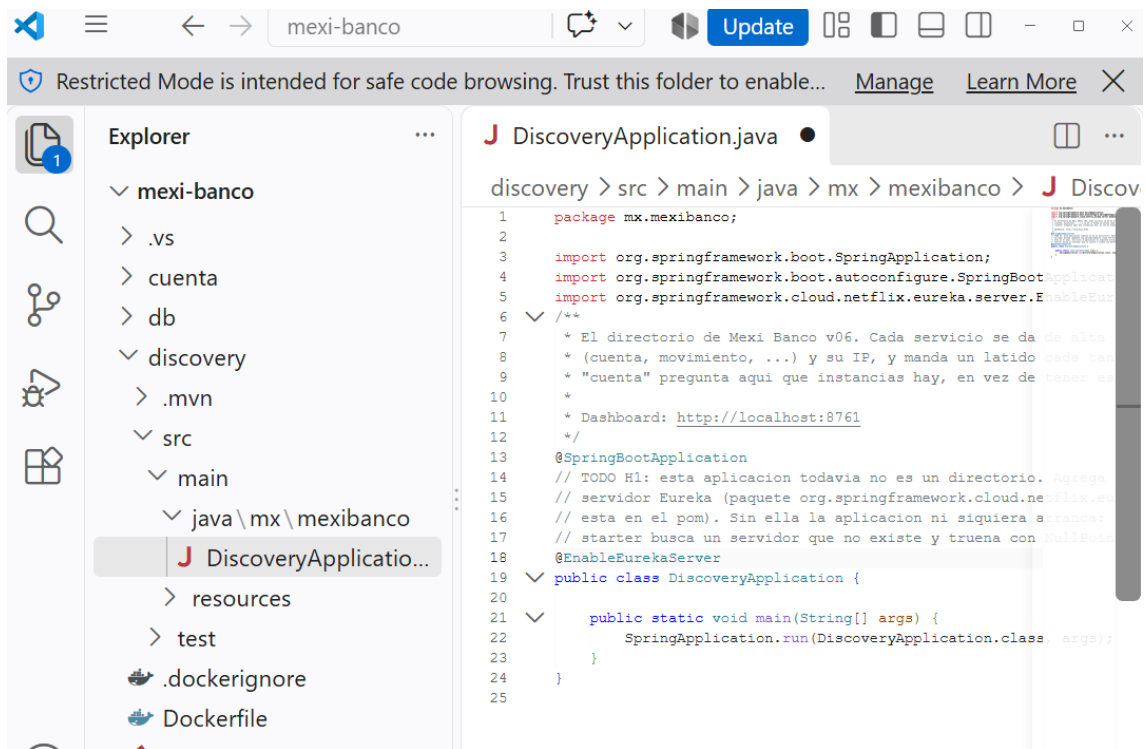


<http://localhost:8761> → no carga.



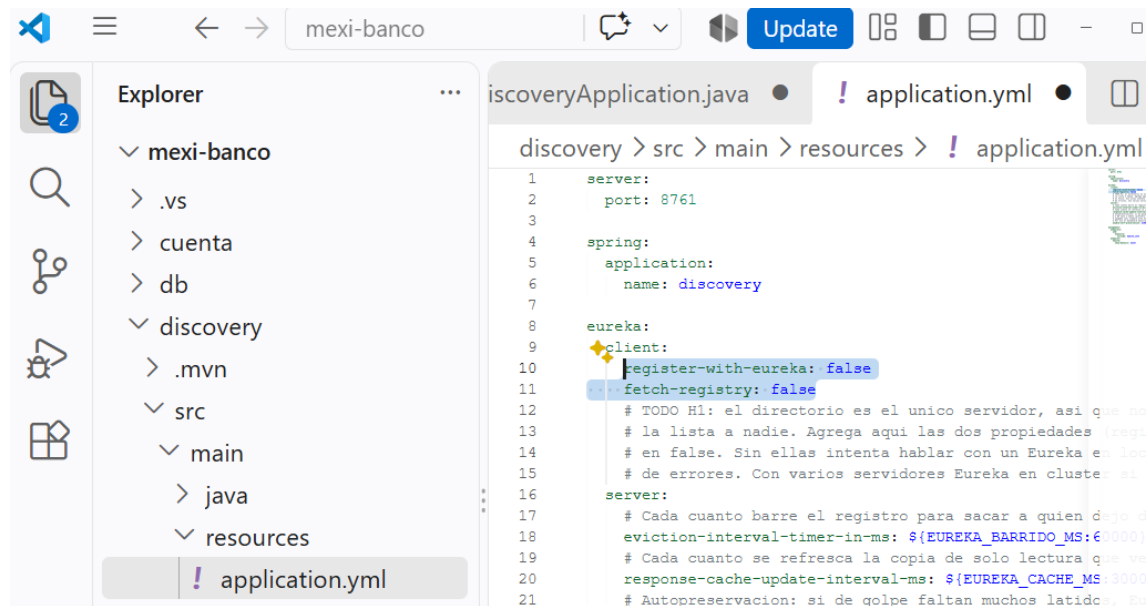
2.1 · Convierte la aplicación en directorio

En `DiscoveryApplication.java` agregué el import de `EnableEurekaServer` y la anotación `@EnableEurekaServer`. Con esto habilité la aplicación discovery como servidor Eureka.



2.2 · Que el directorio no se anote a sí mismo

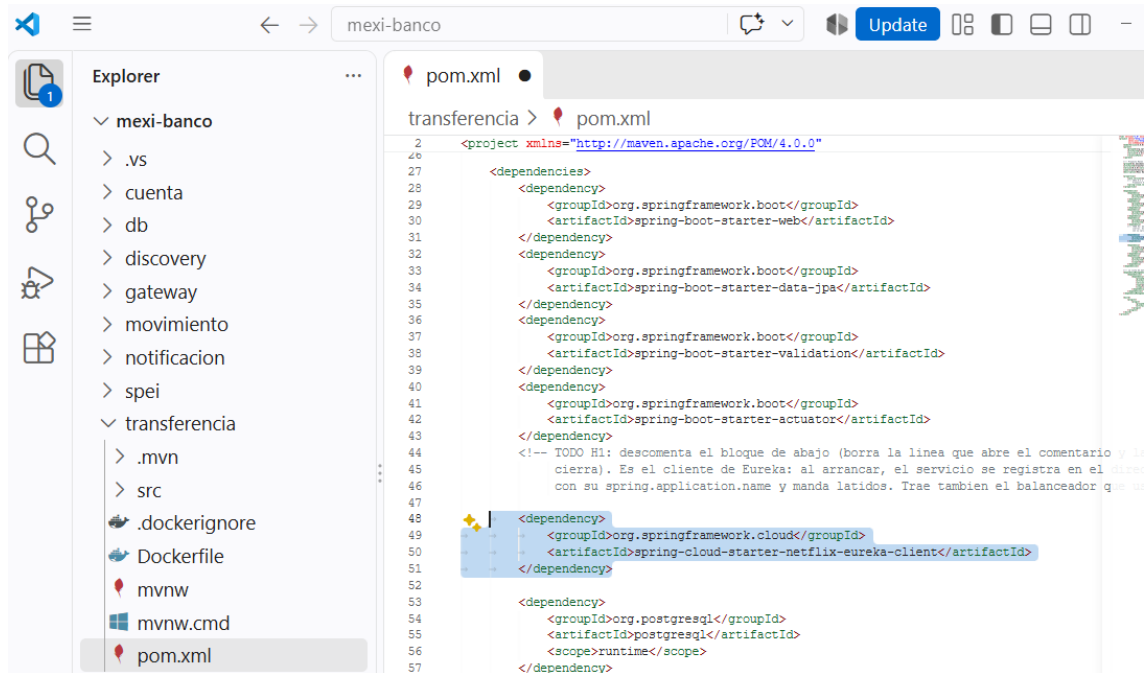
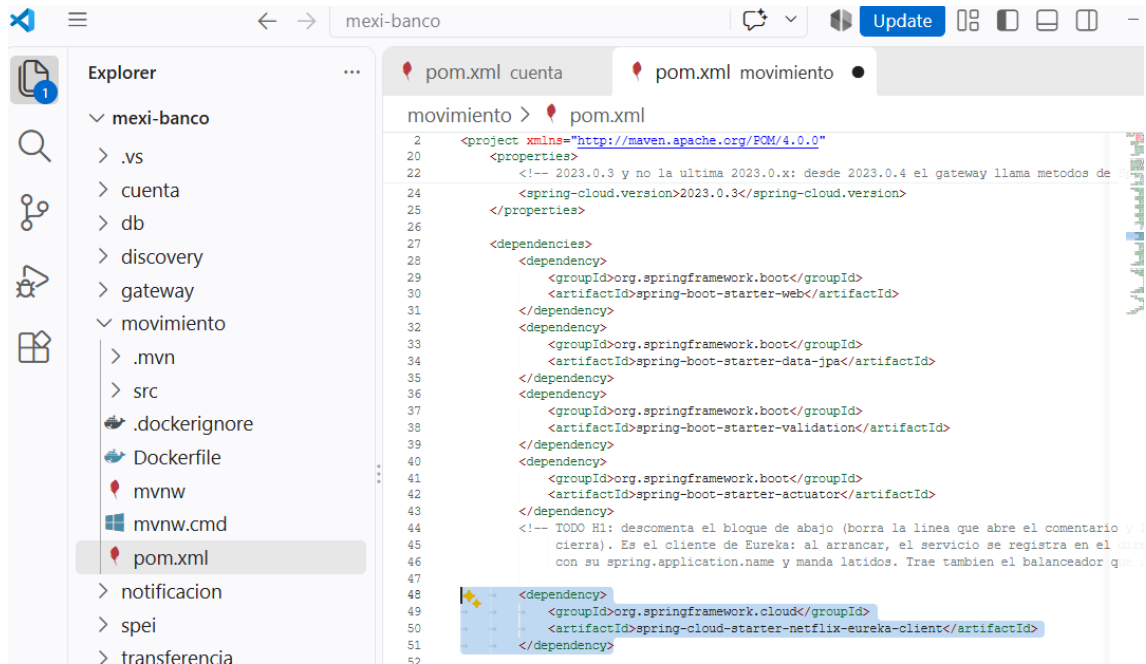
En el application.yml de discovery agregué register-with-eureka: false y fetch-registry: false. Estas opciones evitan que el directorio intente registrarse a sí mismo o consultar otro directorio, porque en este laboratorio utilizamos uno solo.

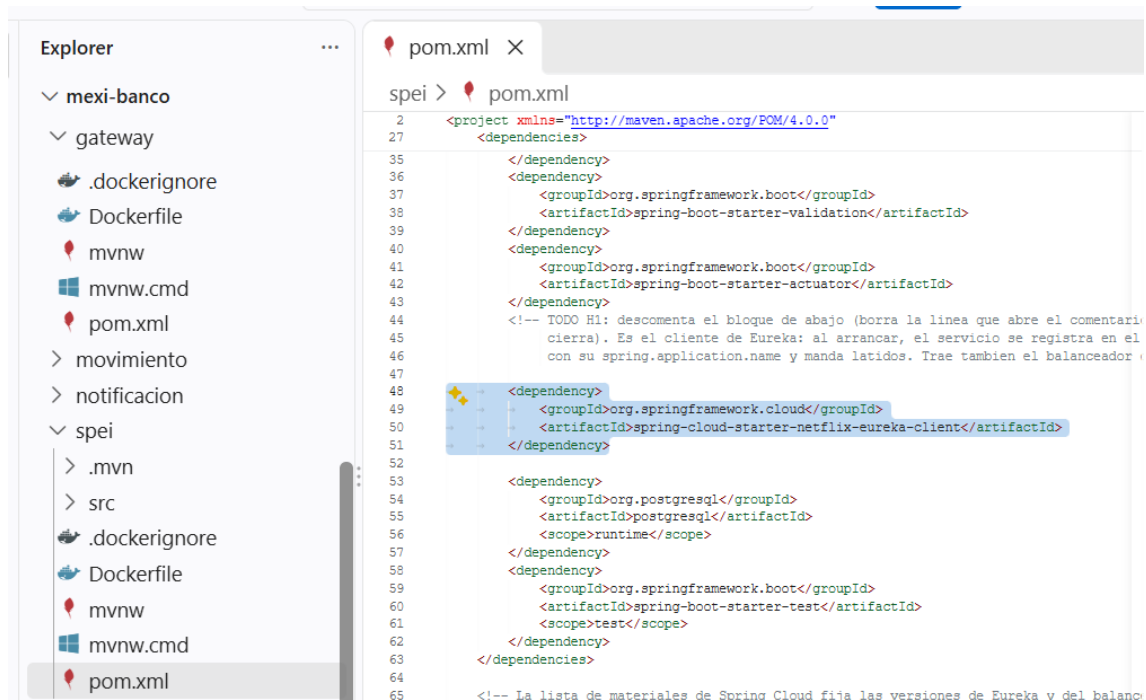
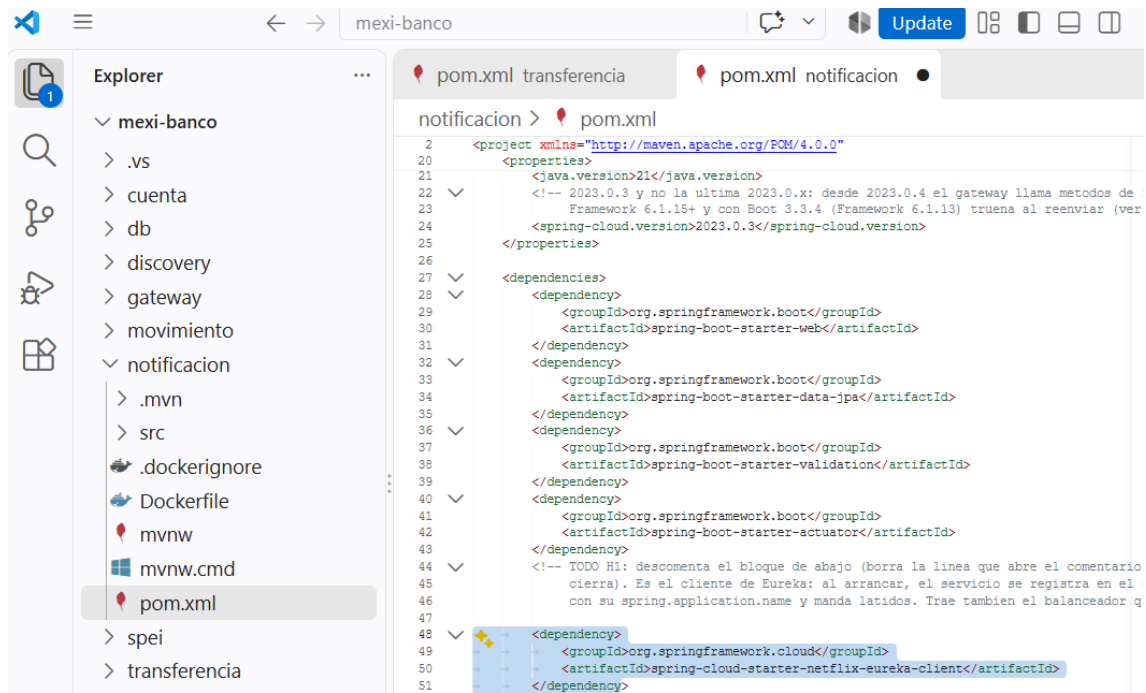


2.3 Registro de los servicios

Inicialmente habilité la dependencia spring-cloud-starter-netflix-eureka-client en cuenta, movimiento, transferencia y notificación. Al revisar el tablero identifiqué que SPEI no aparecía porque su dependencia seguía comentada. El 9 de octubre la habilité, reconstruí el servicio y comprobé su registro en Eureka con estado UP (1).







2.4 · Enciende el directorio en Docker

Descomenté el servicio discovery en docker-compose.yml y su dependencia dentro de depends_on. Así, los cinco servicios esperan a que el directorio esté healthy antes de arrancar. Durante este paso tuve errores de sangría en YAML; corregí la alineación de los bloques y después reconstruí el proyecto.

```
MINGW64/c/Users/cecis/mexi-banco
Debug this Compose error with Gordon - docker ai "help me fix this compose error"

cecis@Asus-Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 2.91kB 0.0s done
#1 DONE 0.0s

#2 [discovery internal] load build definition from Dockerfile
#2 transferring dockerfile: 508B 0.0s done
#2 DONE 0.1s

#3 [spei internal] load build definition from Dockerfile
#3 transferring dockerfile: 457B 0.0s done
#3 DONE 0.1s

#4 [cuenta internal] load build definition from Dockerfile
#4 transferring dockerfile: 457B 0.0s done
#4 DONE 0.1s

#5 [movimiento internal] load build definition from Dockerfile
#5 transferring dockerfile: 457B 0.0s done
#5 DONE 0.2s

#6 [transferencia internal] load build definition from Dockerfile
#6 transferring dockerfile: 457B 0.0s done
#6 DONE 0.1s

#7 [notificacion internal] load build definition from Dockerfile
#7 transferring dockerfile: 457B 0.1s done
#7 DONE 0.1s

#8 [notificacion internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine
#8 DONE 0.1s

#9 [movimiento internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
#9 DONE 0.9s

#10 [notificacion internal] load .dockerignore
#10 transferring context: 49B 0.0s done
#10 DONE 0.1s

#11 [cuenta internal] load .dockerignore
#11 transferring context: 49B 0.0s done
#11 DONE 0.1s

#12 [transferencia internal] load .dockerignore
#12 transferring context: 49B 0.0s done
#12 DONE 0.1s

#13 [notificacion internal] load build context
#13 DONE 0.0s

#14 [spei internal] load .dockerignore
#14 transferring context: 49B 0.0s done
#14 DONE 0.1s
```

```
MINGW64/c/Users/cecis/mexi-banco
#67 51.72 [INFO] --- surefire:3.2.5:test (default-test) @ discovery ---
#67 55.12 [INFO] Tests are skipped.
#67 55.12 [INFO] --- jar:3.4.2:jar (default-jar) @ discovery ---
#67 57.82 [INFO] Building jar: /app/target/discovery-0.6.0.jar
#67 58.31 [INFO] --- spring-boot:3.4.4:repackage (repackage) @ discovery ---
#67 61.30 [INFO] Replacing main artifact /app/target/discovery-0.6.0.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
#67 61.30 [INFO] The original artifact has been renamed to /app/target/discovery-0.6.0.jar.original
#67 61.30 [INFO] BUILD SUCCESS
#67 61.30 [INFO] Total time: 37.257 s
#67 61.31 [INFO] Finished at: 2026-10-03T22:33:37Z
#67 61.31 [INFO]
#67 DONE 67.0s

#60 [discovery stage-1 2/3] WORKDIR /app
#60 CACHED

#77 [discovery stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar
#77 DONE 0.2s

#78 [discovery] exporting to image
#78 exporting layers
#78 exporting layers 3.2s done
#78 exporting manifest sha256:180d18d845ce9980e4afcc1e85092fb6a2353128cc6670f6d37a2b9ae6af71b1 0.0s done
#78 exporting config sha256:180d18d845ce9980e4afcc1e85092fb6a2353128cc6670f6d37a2b9ae6af71b1 0.0s done
#78 exporting attestation manifest sha256:1f1c17b16745471083b8c40913dd39e8c9d031660acce900017e3f4707192a37 0.0s done
#78 exporting manifest list sha256:cc15de5234c22eddb694f32c85991305e668f1caec514ef16e7f788691ac947 0.0s done
#78 exporting manifest list sha256:cc15de5234c22eddb694f32c85991305e668f1caec514ef16e7f788691ac947 0.0s done
#78 naming to docker.io/library/mexi-banco-v06-discovery:latest done
#78 unpacking to docker.io/library/mexi-banco-v06-discovery:latest
#78 unpacking to docker.io/library/mexi-banco-v06-discovery:latest 0.7s done
#78 DONE 4.2s

#79 [discovery] resolving provenance for metadata file
#79 DONE 0.0s
[+] up 13/13
✔ Image mexi-banco-v06-cuenta Built 79.6s
✔ Image mexi-banco-v06-movimiento Built 79.6s
✔ Image mexi-banco-v06-transferencia Built 79.6s
✔ Image mexi-banco-v06-notificacion Built 79.6s
✔ Image mexi-banco-v06-spei Built 79.6s
✔ Image mexi-banco-v06-discovery Built 79.6s
✔ Container mexi-banco-v06-cuenta-1 Running 0.0s
✔ Container mexi-banco-v06-movimiento-1 Running 0.0s
✔ Container mexi-banco-v06-transferencia-1 Running 0.0s
✔ Container mexi-banco-v06-notificacion-1 Running 0.0s
✔ Container mexi-banco-v06-spei-1 Running 0.0s
✔ Container mexi-banco-v06-db-1 Healthy 1.4s
✔ Container mexi-banco-v06-discovery-1 Healthy 16.8s
```

```

cristianus_ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
mexi-banco-v06-cuenta-1	sha256:cddc5382be9c09a21e7ef949ff0eae3a2aa8ba5d4a8f1a0c97a5843b316f62cc	"java -jar app.jar"	cuenta	3 hours ago	Up 3 hours (healthy)	0.0.0.0:8081->
8080/tcp, [::]:8081->8080/tcp						
mexi-banco-v06-db-1	postgres:17-alpine	"docker-entrypoint.s..."	db	4 hours ago	Up 4 hours (healthy)	5432/tcp
mexi-banco-v06-discovery-1	mexi-banco-v06-discovery	"java -jar app.jar"	discovery	3 hours ago	Up 3 hours (healthy)	0.0.0.0:8761->
8761/tcp, [::]:8761->8761/tcp						
mexi-banco-v06-movimiento-1	sha256:945880d50f38df4b3527bcfb7d6476a04ed96c6c23161742bc384333a2e2425	"java -jar app.jar"	movimiento	3 hours ago	Up 3 hours (healthy)	0.0.0.0:8082->
8080/tcp, [::]:8082->8080/tcp						
mexi-banco-v06-notificacion-1	sha256:e6235b4b20425702dc65e2b34850a46054d484fce880e184c275412484e7544	"java -jar app.jar"	notificacion	3 hours ago	Up 3 hours (healthy)	0.0.0.0:8084->
8080/tcp, [::]:8084->8080/tcp						
mexi-banco-v06-spei-1	sha256:b7019a7a5bc01c52be99f1a2b0924fe33c94be7d59773edaa75d41883a66673	"java -jar app.jar"	spei	4 hours ago	Up 4 hours (healthy)	0.0.0.0:8085->
8080/tcp, [::]:8085->8080/tcp						
mexi-banco-v06-transferencia-1	sha256:4d5622bcl585ad750959d92a1d77a824eb88f17ae2b54238220077344663b20e	"java -jar app.jar"	transferencia	3 hours ago	Up 3 hours (healthy)	0.0.0.0:8083->
8080/tcp, [::]:8083->8080/tcp						

2.5 · Compruébalo en el navegador

El tablero de Eureka permite observar las instancias registradas. La API `/eureka/apps/CUENTA` muestra la información de cuenta, como su dirección, puerto, estado y tiempos de renovación y expiración.

Aunque los servicios ya se registraban en Eureka, transferencia todavía no utilizaba el directorio para encontrar cuenta. Su configuración conservaba `CUENTA_URL`: <http://cuenta:8080>, cuyo nombre se resolvía mediante el DNS de Docker. Registrarse en Eureka no cambia automáticamente la manera en que un servicio realiza sus llamadas.

http://localhost:8761

spring Eureka

HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2026-10-09T20:52:42 -0000
Data center	default	Uptime	01:15
		Lease expiration enabled	true
		Renews threshold	0
		Renews (last min)	168

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 12f07d470207:cuenta:8080 , bbe9708b254f:cuenta:8080 , 825681caf8e5:cuenta:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - bbe8b3a0dcdd:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 5dac15254c4e:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 921dc0c68eae:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 5dc4c0e2729b:transferencia:8080

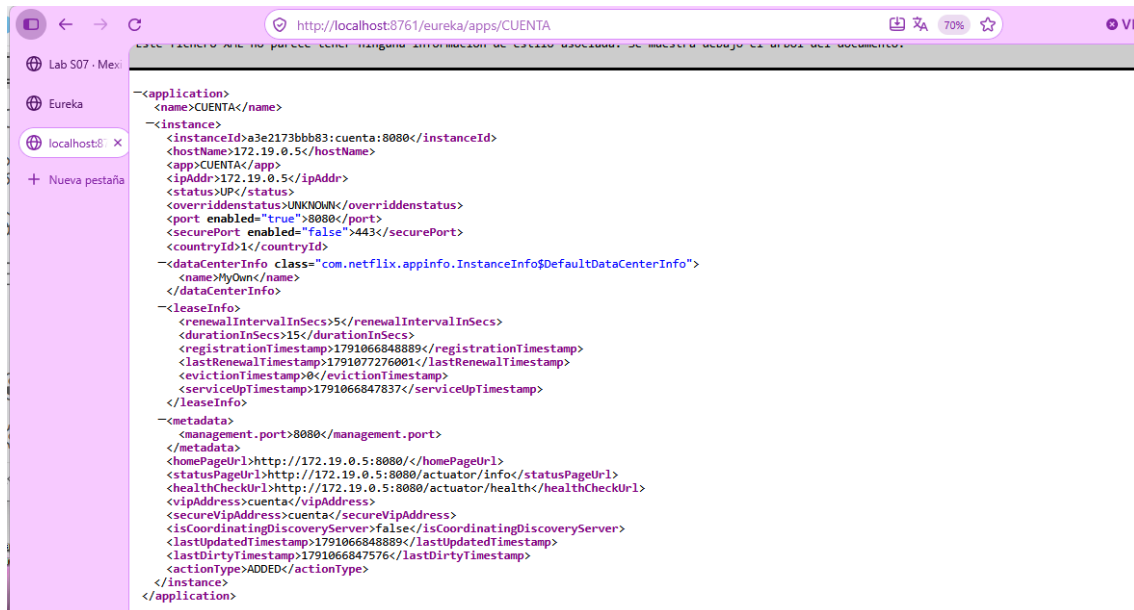
General Info

Name	Value
total-avail-memory	118mb
num-of-cpus	12
current-memory-usage	90mb (76%)
server-uptime	01:15
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/ ,
available-replicas	

Instance Info

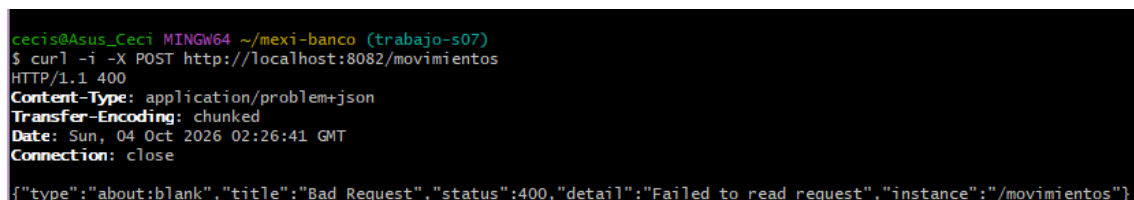
Name	Value
ipAddr	172.19.0.3
status	UP

NOTA: Comprobación del 9 de octubre de 2026, después de corregir el registro de SPEI. CUENTA aparece con UP (3) porque esta captura se tomó después de completar el paso 4. En la comprobación inicial del paso 2.5, CUENTA tenía una sola instancia, UP (1).



Aquí se muestra la instancia a3e2173bbb83:cuenta:8080, con dirección interna 172.19.0.5, puerto 8080 y estado UP. Los valores renewalIntervalInSecs y durationInSecs confirman un heartbeat cada 5 segundos y un plazo de expiración de 15 segundos.

3. El gateway: una sola puerta

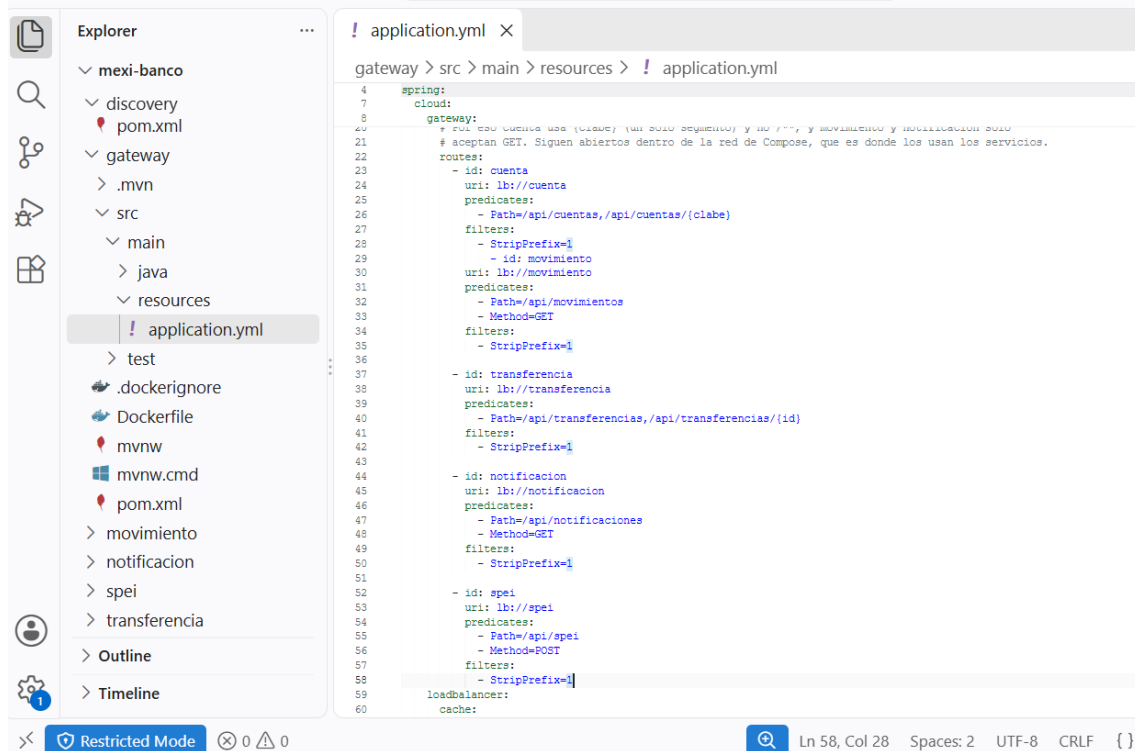


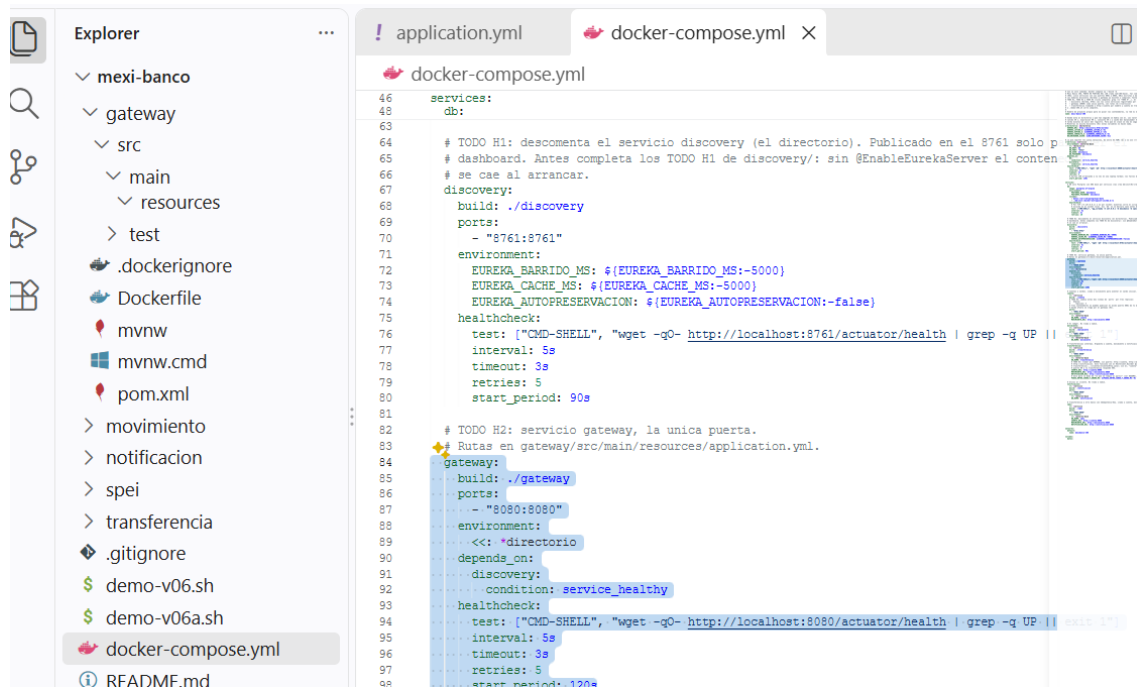
El 3 de octubre de 2026, a las 20:26:41, hora de Ciudad de México, envié un POST directamente a http://localhost:8082/movimientos sin proporcionar datos. El servicio respondió 400 Bad Request. Esto confirma que el endpoint era accesible desde mi computadora, aunque la solicitud no contenía la información necesaria.

3.1 · Escribe las rutas

Configuré el gateway como entrada para los clientes en el puerto 8080. Agregué las rutas de movimiento, transferencia, notificación y SPEI, conservando la ruta de cuenta. Las direcciones lb:// permiten localizar las instancias mediante Eureka, y StripPrefix=1 elimina /api antes de reenviar la solicitud.

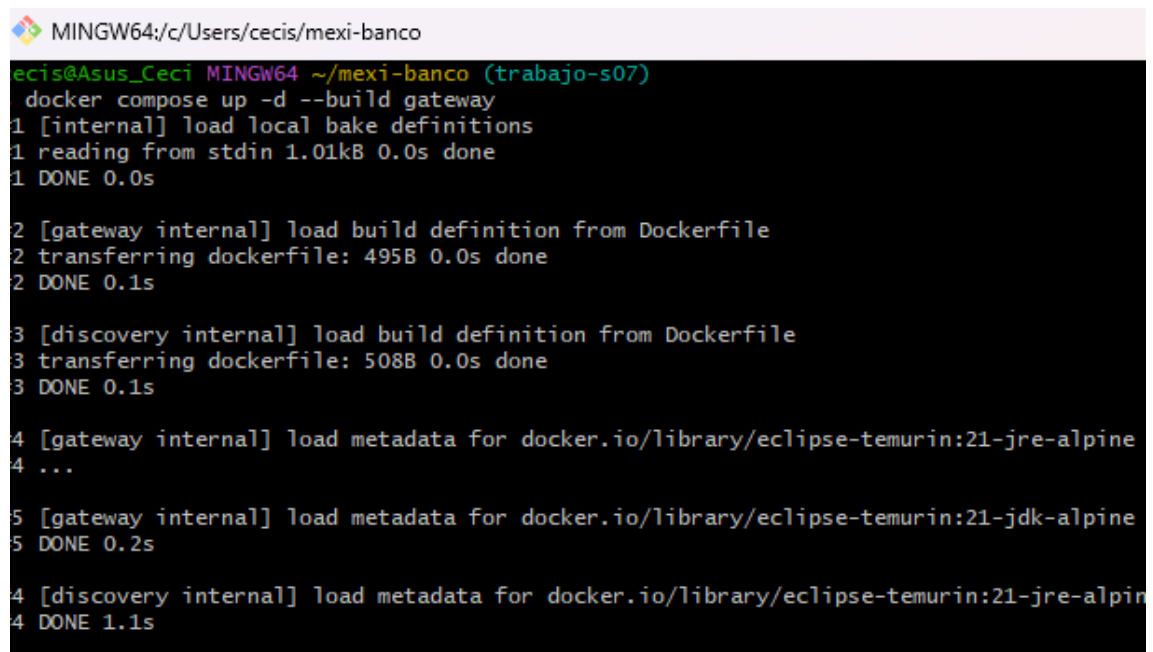
Las rutas de movimiento y notificación permiten únicamente GET, mientras que SPEI permite POST. Así, las operaciones internas para registrar movimientos y notificaciones no quedan disponibles por el gateway.





3.2 · Enciende el gateway

Después habilité el servicio gateway en docker-compose.yml. Durante su arranque apareció un error de configuración YAML en las rutas. Corregí la sangría, reconstruí el gateway y repetí las pruebas.



```
#11 [gateway] resolving provenance for metadata file
#11 DONE 0.0s
[+] up 4/4
  ✓ Image mexi-banco-v06-gateway      Built      33.3s
  ✓ Image mexi-banco-v06-discovery     Built      33.3s
  ✓ Container mexi-banco-v06-discovery-1 Healthy    0.9s
  ✓ Container mexi-banco-v06-gateway-1 Started     1.3s
```

The screenshot shows the Spring Eureka System Status page. The page is divided into several sections:

- Environment:** test, Data center: default.
- Current time:** 2026-10-04T02:58:52+0000.
- Uptime:** 04:25.
- Lease expiration enabled:** true.
- Renews threshold:** 0.
- Renews (last min):** 120.
- THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.**
- DS Replicas:** localhost.
- Instances currently registered with Eureka:**

Application	AMbs	Availability Zones	Status
CLIENTA	n/a (1)	(1)	UP (1) - 6b2c1278b0d35e0c0a5080
GATEWAY	n/a (1)	(1)	UP (1) - 7b1c3b0d0e00 gateway 8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - f6d0e0d0c008 movimiento 8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 56c179b1305 notificacion 8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 2e702e7110e0 transferencia 8080
- General Info:**

Name	Value
total-avail-memory	114mb
num-of-cpus	12
current-memory-usage	40mb (35%)
server-up-time	04:25
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/
available-replicas	
- Instance Info:**

Name	Value
ipAddr	172.19.0.8
status	UP

3.3 · Pruébalo

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: a3e2173bbb83
Content-Type: application/problem+json
Date: Sun, 04 Oct 2026 03:00:11 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-04T03:00:12.768+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"cae64838-94"}
```

Prueba de las rutas — 3 de octubre, 21:00

Consulté `/api/cuentas/000` y obtuve un 404 con el mensaje “No existe la CLABE 000”. La respuesta incluía `X-Instancia: a3e2173bbb83`, por lo que la solicitud sí llegó al servicio cuenta: el error se debía a que la cuenta no existía.

Después envié un POST a `/api/movimientos` y también obtuve un 404. En este caso, el gateway no encontró una ruta que permitiera ese método. Por eso no reenvió la solicitud a movimiento.

```

cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d '{"clabe":"$LAB_CLABE","titular":"S07","saldoInicial":0}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: a3e2173bbb83
Content-Type: application/json
Date: Sun, 04 Oct 2026 03:01:32 GMT

{"id":1,"clabe":"S071791082891","titular":"S07","saldo":0}

```

Creación de mi cuenta — 3 de octubre, 21:01

Creé la cuenta S071791082891 a través del gateway, con titular S07 y saldo inicial 0. La operación respondió 201 Created y devolvió los datos de la cuenta.

```

cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ URL="http://localhost:8080/api/cuentas/$LAB_CLABE"
curl -i "$URL"
echo "$URL"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: a3e2173bbb83
Content-Type: application/json
Date: Sun, 04 Oct 2026 03:03:52 GMT

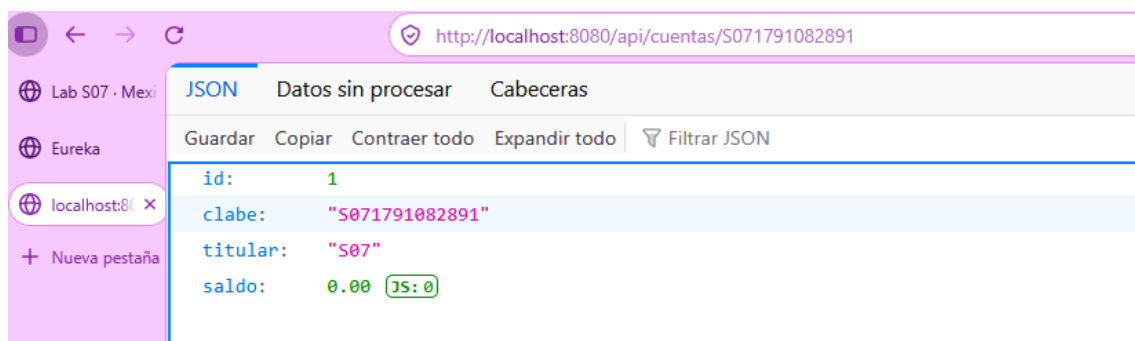
{"id":1,"clabe":"S071791082891","titular":"S07","saldo":0.00}http://localhost:8080/api/cuentas/S071791082891

```

Consulta de mi cuenta — 3 de octubre, 21:03

Consulté la cuenta recién creada mediante el gateway. La respuesta fue 200 OK y mostró saldo 0.00. También abrí la misma dirección en el navegador y comprobé que aparecían los datos en JSON.

El navegador muestra los datos en JSON, con titular S07 y saldo 0.00.



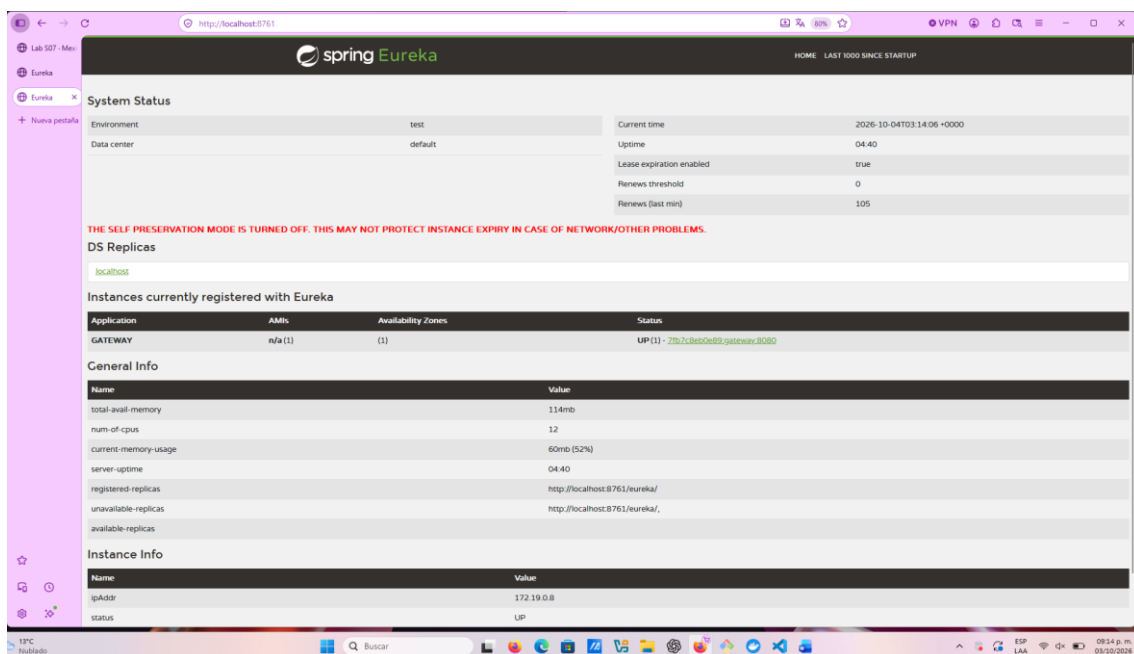
Consulta de la cuenta S071791082891 mediante el gateway en localhost:8080.

Paso 4.1

Modifiqué docker-compose.yml para levantar tres réplicas del servicio cuenta. Reemplacé su bloque ports por deploy con replicas: 3. También retiré los puertos publicados de movimiento, transferencia, notificación y SPEI. Dejé publicados los puertos del gateway y del directorio.

Después de ejecutar docker compose up -d, comprobé que existían cuenta-1, cuenta-2 y cuenta-3. Sus identificadores fueron 8496542147a9, 0b8a822da36a y 9132e1d08d1d, respectivamente. Cada contenedor escuchaba internamente en 8080/tcp, sin publicar ese puerto en mi computadora.

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose up -d
docker ps --filter name=cuenta --format '{{.ID}} {{.Names}} {{.Ports}}'
[+] up 10/10
✓ Container mexi-banco-v06-db-1 Healthy
✓ Container mexi-banco-v06-gateway-1 Running
✓ Container mexi-banco-v06-discovery-1 Healthy
✓ Container mexi-banco-v06-cuenta-1 Started
✓ Container mexi-banco-v06-notificacion-1 Started
✓ Container mexi-banco-v06-cuenta-2 Started
✓ Container mexi-banco-v06-spei-1 Started
✓ Container mexi-banco-v06-transferencia-1 Started
✓ Container mexi-banco-v06-cuenta-3 Started
✓ Container mexi-banco-v06-movimiento-1 Started
8496542147a9 mexi-banco-v06-cuenta-1 8080/tcp
9132e1d08d1d mexi-banco-v06-cuenta-3 8080/tcp
0b8a822da36a mexi-banco-v06-cuenta-2 8080/tcp
```



4.2 · Mira cómo reparte

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: 0b8a822da36a
X-Instancia: 9132e1d08d1d
X-Instancia: 8496542147a9
X-Instancia: 0b8a822da36a
X-Instancia: 9132e1d08d1d
X-Instancia: 8496542147a9
X-Instancia: 0b8a822da36a
X-Instancia: 9132e1d08d1d
X-Instancia: 8496542147a9
```

Comprobación del balanceo — 3 de octubre, aproximadamente 21:16

Consulté mi cuenta nueve veces mediante el gateway y revisé la cabecera X-Instancia. Las respuestas siguieron el orden cuenta-2, cuenta-3 y cuenta-1, repetido tres veces. Cada copia atendió tres consultas. Esto demuestra el reparto por turnos, llamado round robin.

El gateway repartió las consultas entre las tres copias de cuenta. Sin embargo, transferencia todavía utilizaba CUENTA_URL: <http://cuenta:8080>, cuyo nombre se resolvía mediante el DNS de Docker. Por tanto, aún no utilizaba Eureka y su balanceador para repartir las llamadas internas. Ese cambio quedó pendiente para la tarea.

5¿Y si una copia se muere?

No registré una predicción antes de apagar la copia. En esta sección documento los resultados que observé durante el experimento.

El sábado 3 de octubre de 2026 ejecuté 60 consultas a mi cuenta mediante el gateway. Mientras corrían, apagué cuenta-1 de golpe con docker kill a las 21:21:25, hora de Ciudad de México.

```
MINGW64; c:/Users/cecis/mexi-banco

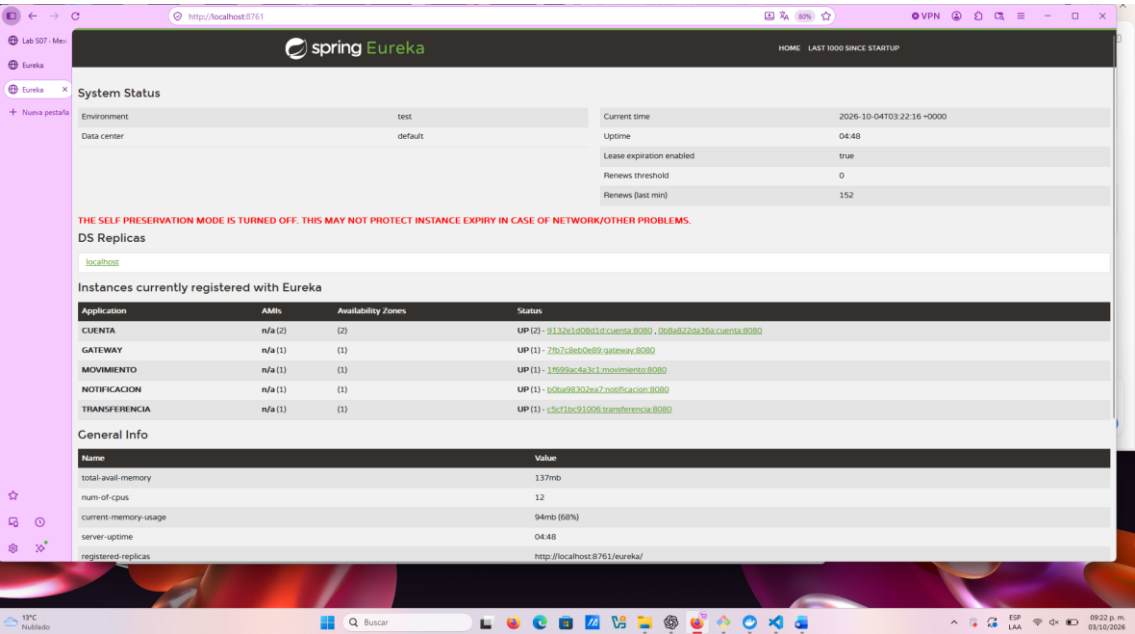
cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ for i in {1..60}; do
  echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
  sleep 1
done
21:21:16 200
21:21:17 200
21:21:18 200
21:21:20 200
21:21:21 200
21:21:22 200
21:21:24 200
21:21:25 200
21:21:27 200
21:21:28 500
21:21:31 200
21:21:33 500
21:21:36 200
21:21:38 200
21:21:39 200
21:21:41 500
21:21:44 200
21:21:45 200
21:21:47 500
21:21:50 200
21:21:51 200
21:21:53 500
21:21:56 200
21:21:57 200
21:21:59 200
21:22:00 500
21:22:04 200
21:22:05 200
21:22:06 200
21:22:08 200
21:22:09 200
21:22:10 200
21:22:12 200
21:22:13 200
21:22:14 200
21:22:16 200
21:22:17 200
21:22:18 200
21:22:20 200
21:22:21 200
21:22:22 200
21:22:24 200
21:22:25 200
21:22:26 200
21:22:28 200
21:22:30 200
21:22:31 200
21:22:32 200
21:22:34 200
21:22:35 200
21:22:36 200
21:22:38 200
21:22:39 200
21:22:40 200
21:22:42 200
21:22:43 200
21:22:44 200
21:22:46 200
21:22:47 200
21:22:48 200
```

De las 60 consultas, 54 respondieron 200 y 6 respondieron 500. El primer error apareció a las 21:21:28 y el último a las 21:22:00, 35 segundos después del apagado. Desde la siguiente consulta, a las 21:22:04, todas respondieron 200 hasta finalizar el ciclo.

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ date +%T
21:21:25
docker kill mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
```

En la captura de Eureka tomada a las 21:22:16, CUENTA mostraba UP (2) y ya no aparecía la instancia 8496542147a9. Esta observación fue 51 segundos después del

apagado. No registré el instante exacto en que desapareció del tablero, por lo que ese tiempo no representa una medición exacta de su retirada.



Las consultas fallaron de manera intermitente porque el gateway todavía podía seleccionar la copia apagada mientras se actualizaban las listas de instancias. Eureka necesita detectar la ausencia de heartbeats y retirar la instancia; después, el gateway y el balanceador deben actualizar su información. Las otras dos copias continuaron atendiendo solicitudes, por eso no fallaron todas las consultas.

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ docker start mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
```

The screenshot shows the Spring Eureka web interface at <http://localhost:8761>. The interface includes a sidebar with navigation links and a main content area with the following sections:

- System Status:**
 - Environment: test
 - Data center: default
 - Current time: 2026-10-04T03:29:05 -0000
 - Uptime: 04:55
 - Lease expiration enabled: true
 - Renewal threshold: 0
 - Renewal (last min): 144
- D5 Replicas:**
 - localhost
- Instances currently registered with Eureka:**

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 9132e1d08d1dcuenta/8080, 0a8e822da35ecuenta/8080, 8496542147a9cuenta/8080
GATEWAY	n/a (1)	(1)	UP (1) - 7b7c8eb0ef80gateway/8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 1f099ac43c1movimiento/8080
NOTIFICACION	n/a (1)	(1)	UP (1) - b05a98102a72notificacion/8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - c5d11c91006transferencia/8080
- General Info:**

Name	Value
total-avail-memory	137mb
num-of-cpus	12
current-memory-usage	87mb (63%)
server-up-time	04:55
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/
available-replicas	
- Instance Info:**

Name	Value
ipAddr	172.19.0.8
status	UP

Después de reiniciar cuenta-1 con docker start, la instancia volvió a registrarse en Eureka. En la captura tomada a las 21:29:05, hora de Ciudad de México, CUENTA aparece nuevamente en UP (3), incluyendo el identificador 8496542147a9. Esto confirma que las tres copias volvieron a estar registradas.

Cierre

El directorio resuelve cómo localizar los servicios y conocer sus instancias registradas. Lo comprobé en Eureka al observar las copias de cuenta y cómo una desapareció después de apagarla.

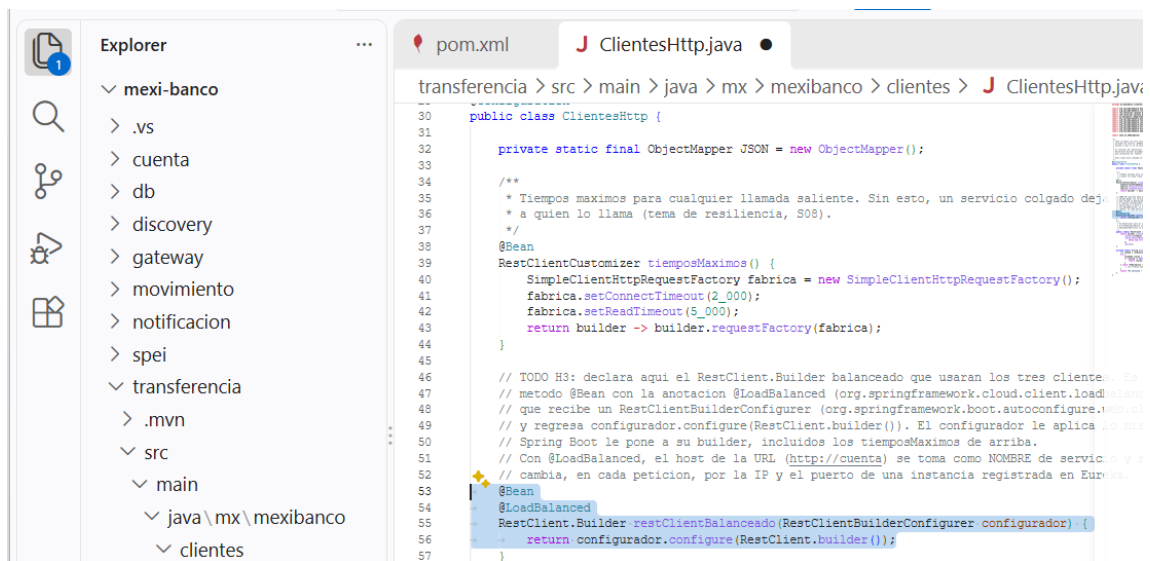
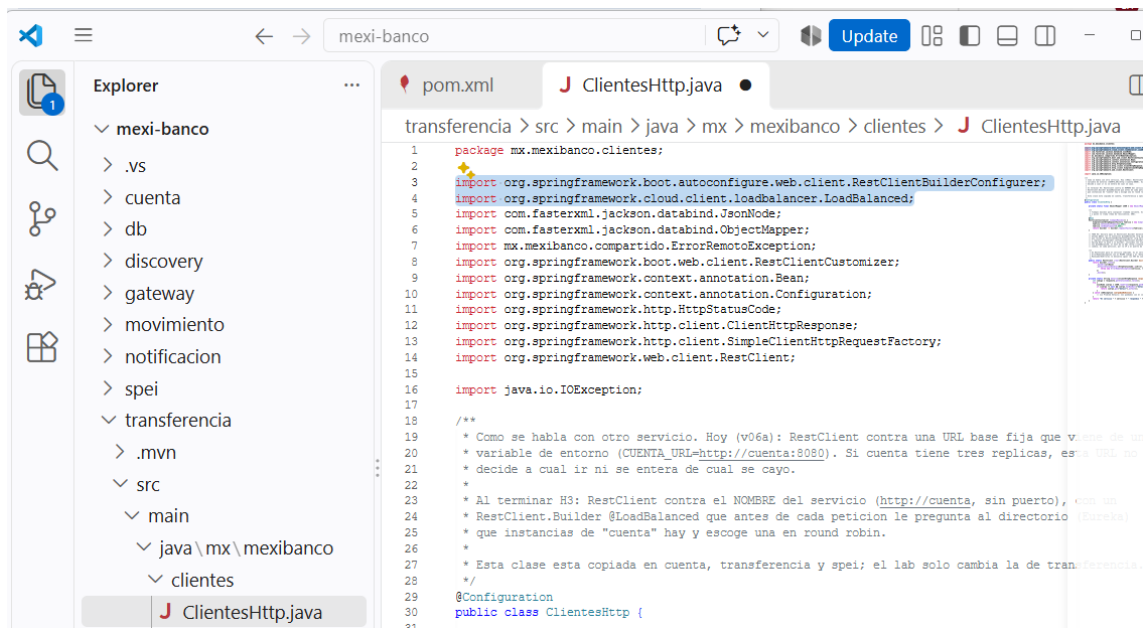
El gateway resuelve tener una entrada común para los clientes y controlar qué rutas y métodos pueden utilizar. Lo comprobé al consultar y crear mi cuenta por el puerto 8080, y al recibir un 404 cuando intenté registrar un movimiento por esa entrada.

El balanceador resuelve repartir las solicitudes entre las copias disponibles. Las nueve cabeceras X-Instancia mostraron que las tres copias respondían por turnos.

Cuando murió una copia vi seis errores 500 entre 60 consultas, porque el gateway todavía podía seleccionarla mientras se actualizaba la información de las instancias. Las otras dos copias siguieron respondiendo y después dejaron de aparecer errores.

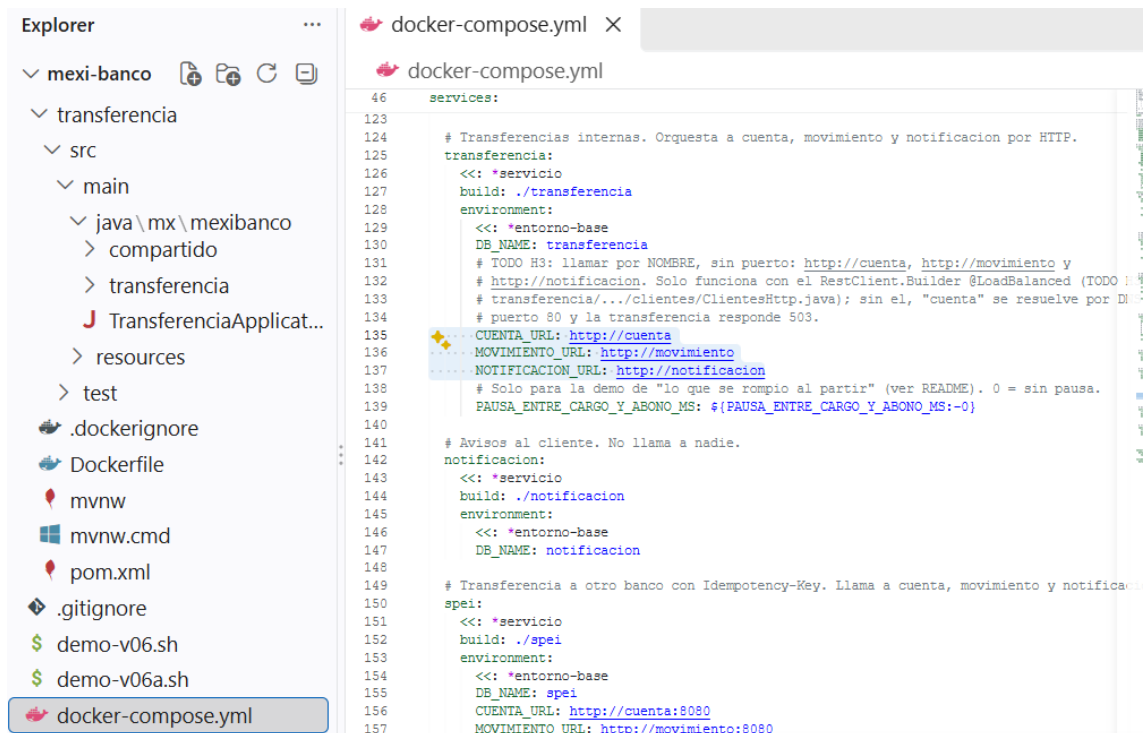
T.1

En ClientesHttp.java del servicio transferencia agregué los imports RestClientBuilderConfigurer y LoadBalanced. Después incorporé el método restClientBalanceado con las anotaciones @Bean y @LoadBalanced. Este builder permite resolver los nombres de los servicios mediante Eureka y repartir las llamadas entre sus instancias.



T.2

En `docker-compose.yml` cambié las URLs de transferencia a <http://cuenta>, <http://movimiento> y <http://notificacion>, sin indicar el puerto. Así, el cliente balanceado puede interpretar cada nombre como un servicio registrado en Eureka.



T3

Reconstruí transferencia, comprobé que todos los contenedores estaban healthy y ejecuté `./demo-v06.sh`. La transferencia de 200 se registró aproximadamente a las 17:14 el 09 de octubre de 2026, hora de Ciudad de México.

```
MINGW64:/c/Users/cecis/mexi-banco
cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose up -d --build transferencia
[+] up 0/1
- Image mexi-banco-v06-transferencia Building
#1 [internal] load local bake definitions
#1 reading from stdin 1.05kB 0.1s done
#1 DONE 0.1s
#2 [transferencia internal] load build definition from Dockerfile
#2 transferring dockerfile: 457B 0.0s done
#2 DONE 0.1s
#3 [discovery internal] load build definition from Dockerfile
#3 transferring dockerfile: 508B 0.0s done
#3 DONE 0.1s
#4 [transferencia internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine
#4 DONE 0.2s
#5 [discovery internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
#5 DONE 1.6s
#6 [discovery internal] load .dockerignore
#6 transferring context: 49B 0.0s done
#6 DONE 0.0s
```

```
[+] up 5/5ferencia] resolving provenance for metadata file
✔ Image mexi-banco-v06-transferencia Built 67.6s
✔ Image mexi-banco-v06-discovery Built 67.6s
✔ Container mexi-banco-v06-db-1 Healthy 5.4s
✔ Container mexi-banco-v06-discovery-1 Healthy 5.4s
✔ Container mexi-banco-v06-transferencia-1 Started 7.3s
```

```
cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED         STATUS
mexi-banco-v06-cuenta-1            mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta     4 hours ago    Up 4 hours (healthy)
mexi-banco-v06-cuenta-2            mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta     4 hours ago    Up 4 hours (healthy)
mexi-banco-v06-cuenta-3            mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta     4 hours ago    Up 4 hours (healthy)
mexi-banco-v06-db-1                postgres:17-alpine                 "docker-entrypoint.s..."            db         4 hours ago    Up 4 hours (healthy)
mexi-banco-v06-discovery-1          sha256:4c73e7b5107a0c3bd7b2ac46e7afc394f7eee158d4f96ca794da05032eb580 "java -jar app.jar"                  discovery  4 hours ago    Up 4 hours (healthy)
mexi-banco-v06-gateway-1            mexi-banco-v06-gateway              "java -jar app.jar"                  gateway    4 hours ago    Up 2 hours (healthy)
mexi-banco-v06-movimiento-1         mexi-banco-v06-movimiento           "java -jar app.jar"                  movimiento 4 hours ago    Up 4 hours (healthy)
mexi-banco-v06-notificacion-1       mexi-banco-v06-notificacion         "java -jar app.jar"                  notificacion 4 hours ago    Up 4 hours (healthy)
mexi-banco-v06-spei-1              mexi-banco-v06-spei                 "java -jar app.jar"                  spei       3 hours ago    Up 3 hours (healthy)
mexi-banco-v06-transferencia-1      mexi-banco-v06-transferencia        "java -jar app.jar"                  transferencia 5 minutes ago  Up 5 minutes (healthy)
```

A continuación, los resultados de correr ./demo-v06.sh (transferencia y casos de error):

```
MINGW64:/c/Users/cecis/mexi-banco
cecis@Asus_Ceci MINGW64 ~/mexi-banco (trabajo-s07)
$ ./demo-v06.sh

== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ==
-> listo en 6 s. Lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA          3 instancia(s)
GATEWAY         1 instancia(s)
MOVIMIENTO      1 instancia(s)
NOTIFICACION    1 instancia(s)
SPEI            1 instancia(s)
TRANSFERENCIA   1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ==
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009158762801","titular":"Ana","saldoInicial":1000}'
{"id":2,"clabe":"002180009158762801","titular":"Ana","saldo":1000}
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

== 2. Abrir la cuenta de Beto con 500 (misma puerta) ==
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009158762802","titular":"Beto","saldoInicial":500}'
{"id":3,"clabe":"002180009158762802","titular":"Beto","saldo":500}
[HTTP 201]

== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ==
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"clabeOrigen":"002180009158762801","clabeDestino":"002180009158762802","monto":200}'
{"id":1,"clabeOrigen":"002180009158762801","clabeDestino":"002180009158762802","monto":200,"fecha":"2026-10-09T23:14:03.677090524Z"}
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-1 GET /cuentas/002180009158762801 atendida por bbe9708b254f
cuenta-2 GET /cuentas/002180009158762802 atendida por 825681caf8e5
cuenta-3 POST /cuentas/002180009158762801/cargos atendida por 12f0/d470207
```

```

MINGW64/c:/Users/cecis/mexi-banco
=== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009158762801","claveDestino":"002180009158762802","monto":200}'
{"id":1,"claveOrigen":"002180009158762801","claveDestino":"002180009158762802","monto":200,"fecha":"2026-10-09T23:14:03.677090524Z"}
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
  cuenta-1 GET /cuentas/002180009158762801 atendida por bbe9708b254f
  cuenta-2 GET /cuentas/002180009158762802 atendida por 825681caf8e5
  cuenta-3 POST /cuentas/002180009158762801/cargos atendida por 12f07d470207
  cuenta-1 POST /cuentas/002180009158762802/abonos atendida por bbe9708b254f
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

=== 4. Consultar esa transferencia ===
$ curl http://localhost:8080/api/transferencias/1
{"id":1,"claveOrigen":"002180009158762801","claveDestino":"002180009158762802","monto":200.00,"fecha":"2026-10-09T23:14:03.677091Z"}
[HTTP 200]

=== 5. Saldos: el dinero no se crea ni se pierde ===
$ curl http://localhost:8080/api/cuentas/002180009158762801
{"id":2,"clave":"002180009158762801","titular":"Ana","saldo":800.00}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009158762802
{"id":3,"clave":"002180009158762802","titular":"Beto","saldo":700.00}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===
$ curl http://localhost:8080/api/movimientos?clave=002180009158762801
[{"id":3,"claveCuenta":"002180009158762801","tipo":"TRANSFERENCIA_ENVIADA","monto":-200.00,"saldoResultante":800.00,"fecha":"2026-10-09T23:14:03.622811Z","referencia":"a 002180009158762802"}, {"id":1,"claveCuenta":"002180009158762801","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-09T23:13:58.215703Z","referencia":"saldo inicial"}]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clave=002180009158762802
[{"id":2,"claveCuenta":"002180009158762802","mensaje":"TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00","fecha":"2026-10-09T23:14:04.864743Z"}]
[HTTP 200]

```

```

MINGW64/c:/Users/cecis/mexi-banco
=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91587628' -d '{"claveOrigen":"002180009158762801","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{"id":1,"idempotencyKey":"spei-demo-91587628","claveOrigen":"002180009158762801","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50,"estado":"ENVIADO"}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91587628' -d '{"claveOrigen":"002180009158762801","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{"id":1,"idempotencyKey":"spei-demo-91587628","claveOrigen":"002180009158762801","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50.00,"estado":"ENVIADO"}
[HTTP 201]
-> mismo id (1 -> 1): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ===
consulta 1 X-Instancia: bbe9708b254f (mexi-banco-v06-cuenta-1)
consulta 2 X-Instancia: 12f07d470207 (mexi-banco-v06-cuenta-3)
consulta 3 X-Instancia: 825681caf8e5 (mexi-banco-v06-cuenta-2)
consulta 4 X-Instancia: bbe9708b254f (mexi-banco-v06-cuenta-1)
consulta 5 X-Instancia: 825681caf8e5 (mexi-banco-v06-cuenta-2)
consulta 6 X-Instancia: bbe9708b254f (mexi-banco-v06-cuenta-1)
consulta 7 X-Instancia: 12f07d470207 (mexi-banco-v06-cuenta-3)
consulta 8 X-Instancia: 825681caf8e5 (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: bbe9708b254f (mexi-banco-v06-cuenta-1)
-> 9 replicas distintas en 9 consultas; 0 veces se repitio la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

=== 9. Los endpoints internos no tienen puerta ===
$ curl -X POST http://localhost:8080/api/cuentas/002180009158762801/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-09T23:14:29.289+00:00","path":"/api/cuentas/002180009158762801/abonos","status":404,"error":"Not Found","requestId":"6fe838fc-1663"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

```

```

=== 10. Errores de negocio, por la puerta, con el mismo código que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009158762801","claveDestino":"002180009158762899","monto":10}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009158762899","instance":"/transferencias","servicio":"cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009158762801","claveDestino":"002180009158762802","monto":999999}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 002180009158762801","instance":"/transferencias","servicio":"cuenta"}
[HTTP 422]

=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco
Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido

```

Una breve explicación:

La demo creó la cuenta de Ana con 1000 y la de Beto con 500. Después de transferir 200 de Ana a Beto, los saldos quedaron en 800 y 700, respectivamente. La transferencia respondió 201 y su consulta posterior respondió 200.

La salida también mostró el reparto de las llamadas internas: cuenta-1 consultó la cuenta de Ana, cuenta-2 consultó la de Beto, cuenta-3 atendió el cargo y cuenta-1 atendió el abono. Esto confirma que transferencia utilizó distintas copias de cuenta para completar la operación.

Después envié un SPEI de 50 dos veces con la misma Idempotency-Key. Ambas respuestas devolvieron el mismo identificador de operación, y el saldo de Ana quedó en 750. Por tanto, el SPEI repetido no produjo un segundo cobro.

Finalmente, la transferencia hacia una cuenta inexistente respondió 404 y la operación con saldo insuficiente respondió 422. Tras estas pruebas, los saldos continuaron en 750 para Ana y 700 para Beto. También comprobé que un intento de abono interno por el gateway respondió 404.

Los resultados coinciden con los esperados en la guía. En esta ejecución no observé un timeout en la transferencia.

Después, ejecuté `docker compose logs cuenta | grep -E 'cargos|abonos'`. Los registros muestran que cuenta-3 atendió el cargo a Ana a las 17:14:02 y cuenta-1 atendió el abono a Beto a las 17:14:03 del 9 de octubre, hora de Ciudad de México. Esto confirma que el cargo y el abono de una misma transferencia fueron procesados por copias distintas.

```
cecils@Asus_Cec1 MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose logs cuenta | grep -E 'cargos|abonos'
cuenta-1 | 2026-10-09T23:14:03.493Z INFO 1 --- [cuenta] [nio-8080-exec-9] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/0021800091587628
02/abonos atendida por bbe9708b254f
cuenta-1 | 2026-10-09T23:14:30.503Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/0021800091587628
01/cargos atendida por bbe9708b254f
cuenta-3 | 2026-10-09T23:14:02.682Z INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/0021800091587628
01/cargos atendida por 12f07d470207
cuenta-3 | 2026-10-09T23:14:14.332Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/0021800091587628
01/cargos atendida por 12f07d470207
```

Las otras líneas corresponden a solicitudes de cargo realizadas durante las pruebas posteriores. Su aparición en los logs no significa que todas hayan descontado dinero, la demo confirmó que los errores no modificaron los saldos.